



Universidade do Minho
Escola de Engenharia

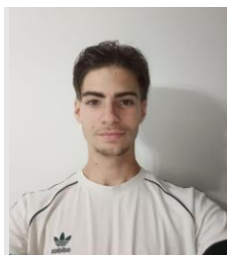
Relatório Trabalho Prático POO (Grupo 17)



Tomé Francisco Amorim Mo-
reira
a111907



Miguel Pedro Ribeiro Correia
a111773



José Pedro Fernandes da Costa
a109807

10 de maio de 2026

Índice

1	Arquitetura de Classes e Decisões de Desenvolvimento	3
1.1	Classe Central do Sistema	3
1.2	Modelação dos Dispositivos	3
1.2.1	Dispositivos On/Off	4
1.2.2	Dispositivos de Medição	4
1.2.3	Dispositivos de Abertura	4
1.2.4	Vantagens da Hierarquia	4
1.3	Modelação dos utilizadores	4
1.3.1	Classe Utilizador	5
1.3.2	Relação entre Utilizadores e Casas	5
1.3.3	Vantagens da Abordagem	5
1.4	Modelação das Ações	5
1.4.1	Ações Simples	6
1.4.2	Ações Compostas	6
1.5	Modelação das Condições	7
1.5.1	Condições Baseadas em Sensores	7
1.5.2	Condições Temporais	7
1.6	Sistema de Automações e Escalonamentos	8
1.6.1	Automações de Estado	8
1.6.2	Automações de Evento	8
1.6.3	Vantagens da Abordagem	9
1.7	Modelação dos Cenários	9
1.8	Modelação do Analisador	9
1.8.1	Representação de Padrões	9
1.8.2	Deteção de Recorrência	10
1.8.3	Criação de Sugestões	10
1.8.4	Vantagens da Abordagem	10
1.9	Modelação da Classe Casa	10
1.9.1	Gestão de Utilizadores	11
1.9.2	Organização das Divisões e Dispositivos	11
1.9.3	Automações e Cenários	11
1.9.4	Integração do Analisador	11
1.9.5	Vantagens da Estrutura Interna	11
1.10	Tratamento de Exceções	12
1.11	Organização da Interface e MVC	12
2	Descrição da aplicação desenvolvida	12
2.1	Gestão de Utilizadores	12
2.2	Gestão de Casas e Divisões	13
2.3	Gestão de Dispositivos	13
2.4	Automações	15
2.4.1	Automações por Sensor	16
2.4.2	Escalonamentos por Horário Exato	16
2.4.3	Escalonamentos por Intervalo	17
2.5	Cenários	17
2.6	Consultas e Estatísticas	18

2.7	Simulação Temporal	19
2.8	Sugestões de Automações	19

1 Arquitetura de Classes e Decisões de Desenvolvimento

A arquitetura da aplicação foi desenvolvida com o objetivo de garantir modularidade, reutilização de código, extensibilidade e separação clara de responsabilidades entre os diferentes componentes do sistema.

A Figura 19 apresenta o diagrama de classes principal da aplicação, ilustrando as relações existentes entre as entidades do sistema.

Nota: Encontra-se disponível na pasta *diagramaClasses* um ficheiro no formato *.vpp*, compatível com o Visual Paradigm, que permite visualizar e analisar o diagrama de classes com maior detalhe e legibilidade.

1.1 Classe Central do Sistema

A classe *DomusControl* funciona como ponto central da aplicação, sendo responsável pela gestão global das casas e dos utilizadores existentes no sistema.

Esta classe agrega todas as entidades principais da aplicação e disponibiliza operações de alto nível relacionadas com autenticação, gestão de casas, consultas globais e persistência do estado da aplicação.

Do ponto de vista arquitetural, a classe aproxima-se do padrão *Facade*, ao concentrar num único componente os principais métodos de interação com o sistema.

Desta forma, várias operações sobre casas, utilizadores e restantes entidades são realizadas através da classe *DomusControl*, reduzindo a dispersão de responsabilidades e simplificando a organização global da aplicação.

Cada casa é representada pela classe *Casa*, responsável pela gestão dos dispositivos, divisões, automações, cenários e permissões de acesso associadas.

A separação entre *DomusControl* e *Casa* permitiu distinguir claramente responsabilidades globais do sistema das responsabilidades locais de cada casa inteligente.

1.2 Modelação dos Dispositivos

Os dispositivos inteligentes foram modelados através de uma hierarquia de herança centrada na classe abstrata *Dispositivo*.

Esta classe contém informação comum a todos os dispositivos, nomeadamente:

- identificador;
- marca;
- modelo;
- consumo energético;
- tempo de utilização;
- número de ativações.

A partir desta classe base foram definidas subclasses especializadas, organizadas segundo o tipo de interação suportada por cada dispositivo.

1.2.1 Dispositivos On/Off

A classe *DispositivoOnOff* representa dispositivos cujo funcionamento assenta num estado binário de ligado ou desligado.

Esta abstração serviu de base para dispositivos como lâmpadas ou colunas de som, permitindo concentrar numa única classe toda a lógica associada à ativação e desativação.

1.2.2 Dispositivos de Medição

A classe *DispositivoMedicao* especializa *DispositivoOnOff*, representando sensores responsáveis pela monitorização do ambiente, como sensores de chuva ou luminosidade.

Para além do comportamento herdado, estes dispositivos mantêm um valor medido que pode ser atualizado ao longo do tempo.

Sempre que ocorre uma alteração relevante nesse valor, o dispositivo notifica automaticamente a casa onde se encontra inserido, permitindo a reavaliação imediata das automações existentes.

Esta solução segue uma arquitetura inspirada no padrão *Observer*, desacoplando os sensores da lógica interna de execução automática.

1.2.3 Dispositivos de Abertura

A classe *DispositivoAbertura* representa dispositivos associados a mecanismos físicos de abertura e fecho, como cortinas ou portões.

Nestes casos, o estado do dispositivo não é apenas binário, podendo frequentemente assumir níveis intermédios de abertura, o que justificou a existência de uma abstração própria.

1.2.4 Vantagens da Hierarquia

Esta organização permitiu reutilizar comportamento comum entre dispositivos semelhantes e especializar apenas as funcionalidades específicas de cada categoria.

Adicionalmente, a utilização de herança e polimorfismo permitiu tratar dispositivos de forma genérica em vários contextos do sistema.

1.3 Modelação dos utilizadores

A gestão de utilizadores foi desenvolvida de forma a permitir que cada utilizador possa estar associado simultaneamente a várias casas do sistema, com diferentes níveis de permissão em cada uma delas.

Um utilizador do *DomusControl* pode, assim, ter acesso a todas as casas das quais seja proprietário ou usufrutuário, sendo possível assumir papéis distintos consoante a casa em causa.

Em particular, um mesmo utilizador pode ser administrador numa determinada casa e possuir apenas permissões básicas noutra, sem qualquer conflito entre esses contextos.

Esta necessidade levou à separação entre a entidade *Utilizador*, que representa a identidade global do utilizador no sistema, e a informação relativa ao acesso concreto a cada habitação.

1.3.1 Classe Utilizador

A classe *Utilizador* armazena a informação base associada a cada utilizador, como nome e endereço de correio eletrónico, sendo este último utilizado como identificador único no sistema.

Esta entidade representa o utilizador de forma independente das casas a que pertence, permitindo reutilizar o mesmo registo em múltiplos contextos.

1.3.2 Relação entre Utilizadores e Casas

A associação entre utilizadores e casas foi modelada através da classe *CasaAcesso*, funcionando como classe associativa entre ambas as entidades.

Esta classe permite representar, para cada par utilizador-casa:

- o utilizador associado;
- a casa a que possui acesso;
- o papel desempenhado nessa casa.

Os diferentes níveis de acesso foram representados através da enumeração *Papel*, distinguindo os dois tipos de utilizador existentes no sistema: administrador e utilizador comum.

Desta forma, tornou-se possível guardar informação adicional sobre a relação entre entidades, algo que não seria adequadamente representado através de uma associação simples.

1.3.3 Vantagens da Abordagem

A solução adotada permitiu representar de forma natural cenários de utilização partilhada, como famílias ou habitações com múltiplos residentes.

Adicionalmente, a separação entre identidade global e permissões locais tornou a arquitetura mais flexível, coerente e preparada para futuras extensões.

1.4 Modelação das Ações

As operações executadas sobre os dispositivos foram modeladas através da interface *Acao*. Cada ação concreta encapsula um comportamento específico do sistema, como ligar, desligar, abrir, fechar ou alterar propriedades de um dispositivo. Desta forma, operações diferentes são representadas através de classes independentes, como por exemplo *Ligar*, *Desligar*, *Abrir* ou *AlterarLuminosidade*.

Esta abordagem permitiu evitar a existência de estruturas extensas baseadas em múltiplos *if/switch-case* para determinar o comportamento a executar sobre cada dispositivo. Em vez disso, cada ação possui a sua própria lógica, tornando o sistema mais modular, extensível e de mais fácil manutenção.

A utilização desta modelação trouxe várias vantagens:

- reutilização das ações em diferentes contextos;
- separação clara entre dispositivos e operações;
- maior facilidade na adição de novas funcionalidades;

- simplificação da implementação de automações e cenários;
- possibilidade de reverter automaticamente determinadas ações.

Foram ainda distinguidos dois tipos principais de ações: ações simples e ações compostas.

1.4.1 Ações Simples

As ações individuais, designadas no projeto como *AcaoSimples*, representam operações direcionadas a um único dispositivo, como ligar uma lâmpada ou abrir uma cortina.

Algumas destas ações implementam adicionalmente a interface *AcaoAutomatizavel*. Esta interface identifica ações passíveis de serem analisadas pelo sistema de detecção de padrões, permitindo posteriormente gerar sugestões automáticas de automações com base no comportamento do utilizador.

Adicionalmente, determinadas ações suportam ainda a operação *inversa()*, permitindo reverter automaticamente alterações previamente realizadas pelo sistema. Esta funcionalidade revelou-se particularmente importante na implementação de automações reativas e escalonamentos por intervalo, onde o sistema necessita restaurar automaticamente o estado anterior dos dispositivos quando determinadas condições deixam de se verificar.

1.4.2 Ações Compostas

Para além das ações individuais, foi também introduzido o conceito de ações compostas, representadas através da interface *AcaoComposta*.

Estas ações permitem aplicar automaticamente uma determinada operação a múltiplos dispositivos em simultâneo, revelando-se particularmente úteis na implementação de automações e cenários.

A principal classe desta componente é *AcaoPorFiltro*, cuja execução se baseia em dois elementos fundamentais:

- um filtro responsável por seleccionar os dispositivos alvo;
- uma fábrica de ações responsável por criar a ação adequada para cada dispositivo seleccionado.

O mecanismo de filtragem foi modelado através da interface funcional *FiltroDispositivo*, que recebe a casa e um dispositivo, devolvendo um valor booleano que indica se o dispositivo deve ou não ser afetado pela ação.

Por sua vez, a interface funcional *FactoryAcao* permite gerar dinamicamente uma ação concreta a partir de um determinado dispositivo.

Esta abordagem tornou possível definir automações genéricas e reutilizáveis, sem necessidade de criar manualmente ações específicas para cada dispositivo existente na casa.

Por exemplo, tornou-se possível representar operações como:

- ligar todas as lâmpadas de uma divisão;
- fechar todas as cortinas da casa;
- desligar todos os dispositivos durante a noite.

A utilização de interfaces funcionais permitiu ainda uma integração natural com expressões *lambda*, contribuindo para uma implementação mais flexível, expressiva e extensível.

1.5 Modelação das Condições

As regras responsáveis pela ativação de comportamentos automáticos no sistema foram modeladas através da interface *Condicao*, permitindo representar de forma abstrata os diferentes critérios que podem desencadear automações ou escalonamentos.

Foram distinguidos dois tipos principais de condições: condições baseadas em sensores e condições temporais.

1.5.1 Condições Baseadas em Sensores

As condições dependentes do estado do ambiente foram modeladas através da interface *CondicaoSensor*.

Este tipo de condição avalia valores medidos por sensores existentes na casa, como luminosidade ou chuva.

Desta forma, tornou-se possível representar comportamentos como:

- ligar iluminação quando a luminosidade desce abaixo de determinado valor;
- fechar cortinas quando começa a chover;

Esta separação permitiu desacoplar a lógica das automações dos dispositivos concretos de medição, facilitando a introdução de novos sensores no futuro.

1.5.2 Condições Temporais

As condições associadas ao tempo interno da casa foram modeladas através da interface *CondicaoTemporal*.

Estas condições utilizam a informação temporal mantida pelo sistema para determinar momentos específicos ou intervalos em que determinadas ações devem ser executadas.

Tal abordagem permitiu representar situações como:

- executar uma ação numa hora definida;
- ativar comportamentos apenas durante determinado período;
- agendar operações recorrentes.

A distinção entre condições temporais e condições de sensores revelou-se particularmente importante para a evolução da arquitetura, permitindo separar dois mecanismos conceitualmente distintos:

- **automações**, desencadeadas por alterações do ambiente;
- **escalonamentos**, dependentes de critérios temporais.

Desta forma, o sistema ficou preparado para futuras extensões sem necessidade de reformular a estrutura base existente.

1.6 Sistema de Automações e Escalonamentos

Após a modelação independente de ações e condições, foi introduzida a classe *Automacao*, responsável por associar uma condição a uma ação a executar automaticamente no contexto de uma casa.

Cada automação possui exatamente uma condição de ativação, sendo esta posteriormente avaliada pelo sistema para determinar se a ação associada deve ou não ser executada.

Durante o desenvolvimento optou-se por utilizar uma única estrutura base para representar tanto automações como escalonamentos. Esta decisão resultou do facto de ambos os mecanismos partilharem a mesma lógica fundamental: verificar uma condição e, caso esta se verifique, executar uma determinada ação.

Desta forma, a distinção entre automação e escalonamento não foi colocada ao nível da classe *Automacao*, mas sim ao nível do tipo de condição utilizado, assunto tratado anteriormente através das interfaces *CondicaoSensor* e *CondicaoTemporal*.

Esta solução permitiu evitar duplicação estrutural e aumentar a reutilização de código, mantendo simultaneamente a flexibilidade necessária para suportar diferentes comportamentos.

1.6.1 Automações de Estado

A classe *AutomacaoEstado* foi criada para representar automatismos cuja condição pode alternar entre verdadeiro e falso ao longo do tempo.

Nestes casos, quando a condição passa a verdadeiro, a ação principal é executada. Quando posteriormente deixa de se verificar, o sistema executa automaticamente a ação inversa, restaurando o estado anterior do dispositivo.

Este modelo revelou-se particularmente útil em situações como:

- ligar iluminação enquanto existe pouca luminosidade;
- fechar cortinas enquanto está a chover;
- ligar iluminação das 20:00 às 23:00;

1.6.2 Automações de Evento

A classe *AutomacaoEvento* foi utilizada para representar comportamentos desencadeados num instante específico, normalmente associados a condições temporais baseadas em *timestamp*.

Neste tipo de automação, quando a condição se verifica, a ação é executada uma única vez, não existindo necessidade de restaurar automaticamente o estado anterior.

Este modelo permitiu representar situações como:

- abrir as cortinas às 07:00;
- desligar iluminação à meia-noite;
- executar uma ação agendada num momento específico.

1.6.3 Vantagens da Abordagem

A separação entre *AutomacaoEstado* e *AutomacaoEvento* permitiu distinguir comportamentos contínuos de comportamentos pontuais, mantendo simultaneamente uma estrutura comum baseada na classe *Automacao*.

Desta forma, a arquitetura tornou-se mais simples, coerente e preparada para futuras extensões do sistema.

1.7 Modelação dos Cenários

Para além das automações, foi também introduzido o conceito de *Cenário*, destinado à execução manual de conjuntos pré-definidos de ações sobre a casa.

Um cenário corresponde a uma agregação lógica de várias ações que podem ser desencadeadas em simultâneo pelo utilizador, permitindo alterar rapidamente o estado global da habitação através de uma única operação.

Cada cenário encapsula um nome identificador e uma coleção de ações previamente configuradas, que são executadas sequencialmente quando o cenário é ativado.

Esta funcionalidade revelou-se particularmente útil para representar situações do quotidiano, como por exemplo:

- **Modo Noite** — desligar as luzes, fechar cortinas e reduzir luminosidade ambiente;
- **Sair de Casa** — desligar dispositivos ativos e fechar as cortinas;
- **Acordar** — abrir cortinas e ligar iluminação suave.

Do ponto de vista arquitetural, os cenários reutilizam diretamente o subsistema de ações já existente, não exigindo novas operações específicas para cada contexto.

Esta decisão permitiu evitar duplicação de lógica e manter coerência entre comandos manuais e ações utilizadas em automações.

Adicionalmente, a modelação adotada tornou simples a criação de novos cenários por parte do utilizador, bastando combinar ações já disponíveis no sistema.

Os cenários funcionam assim como um mecanismo de personalização da casa inteligente, complementando as automações com controlo explícito e imediato por parte do utilizador.

1.8 Modelação do Analisador

Foi igualmente desenvolvido um subsistema de análise comportamental, representado pela classe *Analisador*, com o objetivo de detetar padrões de utilização e sugerir automaticamente novos escalonamentos ao utilizador.

A principal finalidade desta componente consiste em observar ações executadas manualmente de forma recorrente e transformar esses comportamentos repetitivos em sugestões de automatização temporal.

1.8.1 Representação de Padrões

Cada ocorrência relevante é registada através da classe *Padrao*, que agrega a informação necessária para caracterizar um comportamento recorrente:

- hora aproximada da execução;

- tipo de ação realizada;
- dispositivo envolvido;
- utilizador responsável pela ação.

Para reduzir pequenas variações horárias naturais no comportamento dos utilizadores, os instantes registados são agrupados em blocos temporais de 10 minutos.

Desta forma, ações executadas em horários próximos podem ser interpretadas como pertencentes ao mesmo padrão de utilização.

1.8.2 Detecção de Recorrência

O analisador mantém internamente um histórico agregado das ocorrências registadas, contabilizando quantas vezes cada padrão foi observado ao longo de dias distintos.

A contagem por dias independentes revelou-se particularmente importante para evitar falsos positivos resultantes de múltiplas repetições no mesmo dia.

Assim, várias ativações consecutivas num curto intervalo temporal não são interpretadas como um hábito recorrente, enquanto comportamentos repetidos ao longo de diferentes dias ganham relevância estatística.

Adicionalmente, padrões com horários muito próximos podem ser agrupados entre si, permitindo reconhecer rotinas que não ocorrem sempre ao minuto exato.

1.8.3 Criação de Sugestões

Quando um determinado padrão atinge um número mínimo de repetições configurado pelo sistema, o analisador pode gerar automaticamente uma sugestão sob a forma de escalonamento.

Essas sugestões são materializadas através de instâncias de *AutomacaoEvento*, normalmente associadas a uma condição temporal de horário exato.

Por exemplo, se um utilizador ligar repetidamente a mesma iluminação por volta das 19:00 em vários dias diferentes, o sistema poderá sugerir a criação automática desse comportamento.

1.8.4 Vantagens da Abordagem

Esta funcionalidade permitiu aproximar o sistema dos hábitos reais dos utilizadores, reduzindo a necessidade de configuração manual de automatismos.

Ao transformar comportamentos frequentes em sugestões automáticas, o *DomusControl* tornou-se mais adaptativo, inteligente e orientado à utilização quotidiana.

1.9 Modelação da Classe Casa

A classe *Casa* representa a principal unidade funcional do sistema, concentrando toda a informação e comportamento associados a uma habitação inteligente.

Cada instância agrega utilizadores autorizados, dispositivos instalados, divisões, automações, cenários e mecanismos internos de análise comportamental.

Para suportar estas responsabilidades, foram seleccionadas diferentes estruturas de dados de acordo com o tipo de acesso pretendido e com as operações mais frequentes do sistema.

1.9.1 Gestão de Utilizadores

Os acessos à casa foram armazenados através de um conjunto *Set<CasaAcesso>*. Esta escolha permitiu garantir que a mesma associação entre utilizador e casa não é inserida repetidamente, evitando duplicações de permissões e mantendo coerência interna.

1.9.2 Organização das Divisões e Dispositivos

As divisões da casa foram representadas através da estrutura:

- *Map<String, Set<Dispositivo>*

onde o nome da divisão é associado ao conjunto de dispositivos nela presentes. Esta modelação permitiu acesso direto a uma divisão pelo seu nome e, simultaneamente, impedir repetições de dispositivos dentro da mesma divisão. Paralelamente, todos os dispositivos existentes na casa são mantidos em:

- *Map<String, Dispositivo>*

utilizando o identificador único como chave. Esta estrutura revelou-se essencial para operações frequentes como procurar, alterar ou executar ações sobre dispositivos específicos em tempo reduzido, sem necessidade de percorrer todas as divisões.

1.9.3 Automações e Cenários

As automações definidas para a casa foram armazenadas num conjunto *Set<Automacao>*. Esta decisão permitiu evitar duplicação de automatismos equivalentes e simplificar operações de inserção e remoção. Os cenários foram representados através de:

- *Map<String, Cenario>*

utilizando o nome como identificador lógico. Desta forma, tornou-se simples localizar, executar ou substituir cenários previamente definidos pelos utilizadores.

1.9.4 Integração do Analisador

Cada casa possui ainda uma instância própria de *Analisador*, permitindo observar hábitos específicos daquela habitação e gerar sugestões contextualizadas aos seus utilizadores. A associação direta do analisador à casa permitiu manter independência entre diferentes casas do sistema, garantindo que os padrões detetados numa habitação não interferem com outra.

1.9.5 Vantagens da Estrutura Interna

A combinação entre conjuntos e *maps* revelou-se adequada às necessidades do sistema, equilibrando eficiência de acesso, prevenção de duplicados e organização clara da informação. Desta forma, a classe *Casa* tornou-se um núcleo coeso e robusto, capaz de suportar de forma eficiente a gestão diária da habitação inteligente.

1.10 Tratamento de Exceções

Ao longo do desenvolvimento foi adotada uma abordagem baseada em exceções específicas para representar situações de erro relevantes do domínio da aplicação.

Em vez de utilizar erros genéricos, foram criadas classes próprias para casos como utilizadores inexistentes, casas duplicadas, dispositivos não encontrados ou operações inválidas. Esta decisão permitiu tornar o código mais expressivo, facilitar o tratamento diferenciado de erros e fornecer mensagens mais claras ao utilizador durante a interação com o sistema.

1.11 Organização da Interface e MVC

A camada de interação com o utilizador foi estruturada segundo uma abordagem inspirada no padrão *Model-View-Controller*.

A componente *TextUI* concentra toda a lógica de entrada e saída de dados, sendo responsável pela apresentação de menus, leitura de comandos e comunicação textual com o utilizador.

Por sua vez, a classe *SistemaController* recebe as operações solicitadas pela interface e coordena a sua execução junto do modelo de domínio.

O *model* é representado essencialmente pela classe *DomusControl*, onde se encontra concentrada a lógica principal da aplicação e o estado global do sistema.

Esta separação permitiu isolar a interface textual da lógica de negócio, tornando o código mais organizado, reutilizável e simples de manter.

2 Descrição da aplicação desenvolvida

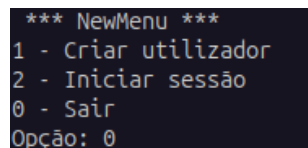
A aplicação desenvolvida consiste num sistema de gestão de casas inteligentes que permite a diferentes utilizadores controlar dispositivos, organizar divisões e definir comportamentos automáticos através de automações, escalonamentos e cenários.

O sistema suporta múltiplos utilizadores, sendo possível a cada utilizador gerir várias casas, com diferentes níveis de permissões (por exemplo, administrador ou utilizador comum).

2.1 Gestão de Utilizadores

A aplicação permite a criação de novos utilizadores e a autenticação através de um sistema de login baseado no email. Após iniciar sessão, o utilizador pode aceder às casas às quais está associado.

A Figura 1 apresenta o menu inicial da aplicação, onde o utilizador pode efetuar autenticação ou criar uma nova conta.



```
*** NewMenu ***
1 - Criar utilizador
2 - Iniciar sessão
0 - Sair
Opção: 0
```

Figura 1: Menu inicial e autenticação de utilizadores

Caso um utilizador tente aceder a uma casa sem permissões, o sistema permite solicitar acesso, podendo este ser atribuído com diferentes papéis.

A Figura 2 ilustra a situação em que um utilizador tenta aceder a uma casa para a qual ainda não possui permissões atribuídas.

```
*** NewMenu ***
1 - Criar utilizador
2 - Iniciar sessão
0 - Sair
Opção: 2
Email do utilizador: diogo@email.com
Sessão iniciada.

*** NewMenu ***
1 - Criar casa
2 - Selecionar casa
0 - Sair
Opção: 2
Nome da casa: CasaVida
Não tens acesso a esta casa.
Queres pedir acesso? (s/n): s
Papel (ADMIN / USER): ADMIN
Acesso atribuído.
Casa selecionada com sucesso.
```

Figura 2: Pedido de acesso a uma casa não atribuída

2.2 Gestão de Casas e Divisões

Cada utilizador pode criar e gerir várias casas. Dentro de cada casa, é possível definir várias divisões (por exemplo, sala, cozinha ou quarto), que servem para organizar os dispositivos existentes.

A Figura 3 apresenta o menu inicial de seleção e gestão de casas associadas ao utilizador autenticado.

```
*** NewMenu ***
1 - Criar casa
2 - Selecionar casa
0 - Sair
Opção: 2
```

Figura 3: Menu de seleção e gestão de casas

Após selecionar uma casa, o utilizador pode gerir as respetivas divisões, criando novas divisões ou consultando as já existentes.

A Figura 4 ilustra o menu de gestão de divisões da aplicação.

```
*** NewMenu ***
1 - Adicionar divisão
2 - Listar divisões
0 - Sair
Opção: 2
```

Figura 4: Gestão de divisões de uma casa

2.3 Gestão de Dispositivos

O sistema suporta diferentes tipos de dispositivos inteligentes, incluindo:

- Lâmpadas, com controlo de luminosidade e cor;
- Colunas de som, com controlo de volume;

- Dispositivos de abertura, como cortinas e portões de garagem;
- Sensores, como sensores de chuva e de luminosidade.

A Figura 5 apresenta o menu principal de gestão de dispositivos, onde o utilizador pode visualizar os dispositivos, adicionar novos dispositivos ou seleccionar um dispositivo específico para interação.

```
*** NewMenu ***
1 - Adicionar Dispositivo
2 - Seleccionar Dispositivo
3 - Listar Dispositivos
0 - Sair
Opção: 2
```

Figura 5: Menu de gestão de dispositivos

Através deste menu, o utilizador pode também listar os dispositivos existentes numa divisão ou criar novos dispositivos. A Figura 6 apresenta o menu de listagem de dispositivos disponíveis, enquanto a Figura 7 mostra o menu utilizado para criação de novos dispositivos inteligentes.

```
*** NewMenu ***
1 - Listar as lâmpadas
2 - Listar as colunas de som
3 - Listar as cortinas
4 - Listar os portões de garagem
5 - ---
6 - Listar os sensores de pluviosidade
7 - ---
8 - Listar todos os dispositivos
0 - Sair
```

Figura 6: Listagem de dispositivos existentes

```
*** NewMenu ***
1 - Criar lâmpada
2 - Criar coluna de som
3 - Criar cortina
4 - Criar portão de garagem
5 - Criar relé
6 - Criar sensor de pluviosidade
7 - Criar sensor de luminosidade
0 - Sair
```

Figura 7: Menu de criação de dispositivos

Após seleccionar um dispositivo, o sistema apresenta um menu intermédio que permite consultar. A Figura 8 apresenta este menu de interação.

```
*** NewMenu ***
1 - Listar dispositivo
2 - Executar uma ação
0 - Sair
Opção: 2
```

Figura 8: Menu de interação de um dispositivo

Cada dispositivo possui um conjunto de funcionalidades específicas que dependem do seu tipo. De forma a modelar estas diferenças, os dispositivos foram organizados em diferentes subclasses.

Os dispositivos do tipo *On/Off*, como lâmpadas e colunas de som, permitem operações de ligar e desligar, podendo ainda disponibilizar funcionalidades adicionais, como ajuste de luminosidade, alteração de cor ou controlo de volume.

Os sensores pertencem à categoria de dispositivos de medição, sendo responsáveis pela recolha de informação do ambiente, como luminosidade ou deteção de chuva. Embora estes dispositivos possam também suportar operações de ligar e desligar, a sua principal função consiste na monitorização e disponibilização de valores associados ao meio envolvente.

Por outro lado, os dispositivos de abertura, como cortinas e portões de garagem, não seguem o paradigma tradicional de ligar/desligar. Em vez disso, suportam operações específicas como abrir, fechar ou definir um determinado grau de abertura.

Quando o utilizador escolhe executar ações sobre um dispositivo, o sistema apresenta dinamicamente apenas as operações compatíveis com o tipo selecionado. A Figura 9 mostra um exemplo do menu de ações disponível para uma lâmpada inteligente.

```
*** NewMenu ***
1 - Ligar
2 - Desligar
3 - Alterar luminosidade
4 - Alterar cor
0 - Sair
Opção: 0
```

Figura 9: Menu de ações disponíveis para um dispositivo

Esta separação permite uma melhor organização do sistema e garante que cada tipo de dispositivo apenas suporta as operações que fazem sentido no seu contexto.

Esta abordagem baseia-se no uso de especialização e polimorfismo, permitindo tratar dispositivos de forma genérica quando possível, mantendo ao mesmo tempo comportamentos específicos para cada tipo.

2.4 Automações

Uma das principais funcionalidades da aplicação é a criação de automações, que permitem executar ações automaticamente com base em determinadas condições. Foram implementados três tipos principais de automações:

A Figura 10 apresenta o menu principal de automações da aplicação. Através deste menu, o utilizador pode criar automações baseadas em sensores, criar escalonamentos por horário exato ou por intervalo, bem como listar todas as automações e escalonamentos existentes no sistema.

```
*** NewMenu ***
1 - Criar automação por sensor
2 - Criar escalonamento por horário exato
3 - Criar escalonamento por intervalo
4 - Listar automações e escalonamentos
0 - Sair
Opção: 0
```

Figura 10: Menu principal de automações

2.4.1 Automações por Sensor

Estas automações são ativadas com base nos valores medidos por sensores. O utilizador pode definir uma condição (por exemplo, valor superior ou inferior a um determinado limite) e um conjunto de ações a executar quando essa condição se verifica.

Como exemplo, considere-se uma casa equipada com um sensor de pluviosidade. Quando o sensor deteta que está a chover, pode ser automaticamente desencadeado um conjunto de ações, como fechar todas as cortinas da casa, tal como ilustrado na Figura 11.

Figura 11: Execução de uma automação baseada em sensor de pluviosidade

```
id = C3, marca = Ikea, modelo = Smart, consumoPorHora = 2.0, tempoAtivo = 0.06, numAtivacoes = 2, abertura(%) = 0
id = C4, marca = Ikea, modelo = Basic, consumoPorHora = 2.0, tempoAtivo = 0.06, numAtivacoes = 2, abertura(%) = 0
```

(a) Estado durante chuva

```
id = C3, marca = Ikea, modelo = Smart, consumoPorHora = 2.0, tempoAtivo = 0.09, numAtivacoes = 3, abertura(%) = 100
id = C4, marca = Ikea, modelo = Basic, consumoPorHora = 2.0, tempoAtivo = 0.09, numAtivacoes = 3, abertura(%) = 100
```

(b) Estado após cessar a chuva

A avaliação destas automações é realizada automaticamente através da utilização do padrão *Observer*. Os dispositivos de medição, como os sensores, notificam o sistema sempre que ocorre uma alteração no valor observado, permitindo que as condições das automações sejam reavaliadas de forma imediata.

Quando a condição deixa de se verificar (ou seja, quando deixa de chover), o sistema é capaz de reverter automaticamente as ações anteriormente executadas, restaurando o estado inicial dos dispositivos.

Destaca-se, assim, a capacidade do sistema de gerir de forma inteligente o comportamento dos dispositivos, através da utilização de ações com inversa, garantindo consistência e adaptação dinâmica às condições do ambiente.

2.4.2 Escalonamentos por Horário Exato

Permitem executar automaticamente um conjunto de ações a uma hora específica do dia. Este tipo de escalonamento é particularmente útil para tarefas rotineiras, permitindo automatizar comportamentos recorrentes sem necessidade de intervenção do utilizador.

Por exemplo, é possível criar um escalonamento que, diariamente às 08:00, abre o portão da garagem e as cortinas da casa, preparando o ambiente para o início do dia, tal como ilustrado na Figura 12.

Figura 12: Execução de um escalonamento baseada em horário (08:00)

```
id = C1, marca = Ikea, modelo = Smart, consumoPorHora = 3.0, tempoAtivo = 0.0, numAtivacoes = 0, abertura(%) = 0
id = C2, marca = Ikea, modelo = Basic, consumoPorHora = 2.0, tempoAtivo = 0.0, numAtivacoes = 0, abertura(%) = 0
id = P1, marca = Somfy, modelo = AutoGate, consumoPorHora = 5.0, tempoAtivo = 0.0, numAtivacoes = 0, abertura(%) = 0
```

(a) Estado antes da execução (antes das 08:00)

```
id = C1, marca = Ikea, modelo = Smart, consumoPorHora = 3.0, tempoAtivo = 0.05, numAtivacoes = 1, abertura(%) = 100
id = C2, marca = Ikea, modelo = Basic, consumoPorHora = 2.0, tempoAtivo = 0.05, numAtivacoes = 1, abertura(%) = 100
id = P1, marca = Somfy, modelo = AutoGate, consumoPorHora = 5.0, tempoAtivo = 0.05, numAtivacoes = 1, abertura(%) = 100
```

(b) Estado após execução do escalonamento (08:00)

A execução destes escalonamentos é realizada com base na evolução do tempo interno da casa, sendo automaticamente avaliados sempre que o tempo é atualizado. Desta forma, o sistema garante que as ações são desencadeadas no momento correto.

Destaca-se a utilidade deste tipo de escalonamento na simplificação de rotinas diárias, contribuindo para maior conforto e eficiência na utilização do sistema.

2.4.3 Escalonamentos por Intervalo

Neste tipo de escalonamento, as ações são ativadas durante um intervalo de tempo definido. No início do intervalo, as ações são executadas automaticamente, sendo posteriormente revertidas no final do mesmo.

Por exemplo, é possível definir um escalonamento que, entre as 20:00 e as 23:00, mantém ligadas apenas as luzes da sala, como ilustrado na Figura 13.

Figura 13: Execução de um escalonamento baseado em intervalo (20:00 - 23:00)

```
id = L6, marca = Philips, modelo = Warm, consumoPorHora = 9.0,
tempoAtivo = 0.0, numAtivacoes = 1, ligado = true, intensidade
= 50, temperaturaCor = 2700
id = L7, marca = Xiaomi, modelo = Soft, consumoPorHora = 7.0, t
empoAtivo = 0.0, numAtivacoes = 0, ligado = false, intensidade
= 0, temperaturaCor = 2700
```

(a) Estado durante o intervalo ativo

```
id = L6, marca = Philips, modelo = Warm, consumoPorHora = 9.0,
tempoAtivo = 3.0, numAtivacoes = 2, ligado = false, intensidade
= 0, temperaturaCor = 2700
id = L7, marca = Xiaomi, modelo = Soft, consumoPorHora = 7.0, t
empoAtivo = 0.0, numAtivacoes = 0, ligado = false, intensidade
= 0, temperaturaCor = 2700
```

(b) Estado após o término do intervalo

De notar que apenas a lâmpada identificada pelo ID L6 é afetada por este escalonamento, uma vez que é a única associada à divisão “sala”. Este comportamento resulta da aplicação de filtros que permitem restringir as ações a um subconjunto específico de dispositivos. Adicionalmente, o sistema garante a reversão automática das ações no final do intervalo definido. Assim, ao terminar o período temporal, os dispositivos voltam ao estado anterior, assegurando consistência no comportamento do sistema ao longo do tempo.

2.5 Cenários

A Figura 14 apresenta o menu principal de gestão de cenários, onde o utilizador pode criar novos cenários, listar os cenários existentes ou executar um cenário previamente definido.

```
*** NewMenu ***
1 - Criar cenário
2 - Listar cenários
3 - Executar cenários
0 - Sair
Opção: 0
```

Figura 14: Menu de gestão de cenários

Os cenários correspondem a conjuntos de ações previamente definidos que podem ser executados manualmente pelo utilizador.

Esta funcionalidade permite simplificar a execução de múltiplas ações em simultâneo, facilitando a criação de rotinas ou ambientes específicos com apenas um comando.

Por exemplo, um cenário denominado “Acordar” pode incluir ações como abrir as cortinas, ligar a luz do quarto e iniciar a reprodução de música, preparando automaticamente o ambiente para o início do dia, tal como ilustrado na Figura 15.

Figura 15: Execução do cenário “Acordar”

```
id = S4, marca = JBL, modelo = Party, consumoPorHora = 20.0, tempoAtivo = 49.0, numAtivacoes = 14, ligado = false, volume = 0
id = C7, marca = Ikea, modelo = Smart, consumoPorHora = 3.0, tempoAtivo = 0.0, numAtivacoes = 0, abertura(%) = 0
id = C8, marca = Ikea, modelo = Basic, consumoPorHora = 2.0, tempoAtivo = 0.5599999999999999, numAtivacoes = 14, abertura(%) = 0
id = L9, marca = Xiaomi, modelo = Soft, consumoPorHora = 7.0, tempoAtivo = 105.0, numAtivacoes = 14, ligado = false, intensidade = 0, temperaturaCor = 2700
```

(a) Estado antes da execução do cenário

```
id = S4, marca = JBL, modelo = Party, consumoPorHora = 20.0, tempoAtivo = 49.0, numAtivacoes = 15, ligado = true, volume = 50
id = C7, marca = Ikea, modelo = Smart, consumoPorHora = 3.0, tempoAtivo = 0.04, numAtivacoes = 1, abertura(%) = 100
id = C8, marca = Ikea, modelo = Basic, consumoPorHora = 2.0, tempoAtivo = 0.6, numAtivacoes = 15, abertura(%) = 100
id = L9, marca = Xiaomi, modelo = Soft, consumoPorHora = 7.0, tempoAtivo = 105.0, numAtivacoes = 15, ligado = true, intensidade = 50, temperaturaCor = 2700
```

(b) Estado após execução do cenário

Através da Figura 15, é possível observar o funcionamento do cenário, verificando-se a alteração simultânea de vários dispositivos conforme definido.

Destaca-se, assim, a utilidade dos cenários na simplificação da interação com o sistema, permitindo ao utilizador executar múltiplas ações de forma rápida, intuitiva e eficiente.

2.6 Consultas e Estatísticas

A aplicação disponibiliza um conjunto de consultas que permitem analisar o comportamento do sistema e obter informação relevante sobre a utilização dos dispositivos e das casas.

Em particular, foram implementadas as seguintes funcionalidades:

- Identificação da casa com maior consumo energético;
- Determinação dos três dispositivos mais utilizados numa determinada casa, tendo em conta o tempo de utilização e, em caso de empate, o número de ativações;
- Identificação das três divisões, juntamente com a respetiva casa, que possuem o maior número de dispositivos;
- Cálculo do consumo médio por divisão, permitindo analisar a distribuição do consumo dentro de uma casa;
- Identificação do utilizador com maior número de casas nas quais possui o papel de administrador.

A Figura 16 apresenta o menu de consultas disponível na aplicação, onde o utilizador pode seleccionar e executar as diferentes análises implementadas.

```
*** NewMenu ***
1 - Casa com maior consumo
2 - Top 3 divisões com mais dispositivos
3 - Utilizador com mais casas (admin)
4 - Consumo médio por divisão (casa atual)
5 - Top 3 dispositivos mais usados (casa atual)
0 - Sair
Opção: 0
```

Figura 16: Menu de consultas e estatísticas

Estas consultas permitem ao utilizador obter uma visão detalhada sobre o funcionamento do sistema, contribuindo para uma gestão mais eficiente dos recursos e dos dispositivos.

2.7 Simulação Temporal

O sistema inclui ainda uma funcionalidade de simulação temporal, que permite avançar o tempo da casa. Esta funcionalidade é particularmente útil para testar o funcionamento dos escalonamentos, permitindo verificar se estas são executadas corretamente ao longo do tempo.

A Figura 17 apresenta a interface utilizada para avançar o tempo, onde o utilizador define os dias, horas e minutos que pretende simular.

```
Tempo atual da casa: 2026-05-06 09:20
Introduza o tempo a avançar (use 0 se não aplicável).
Dias a avançar: 0
Horas a avançar: 3
Minutos a avançar: 4
Novo tempo da casa: 2026-05-06 12:24
Automações temporais foram avaliadas automaticamente.
```

Figura 17: Simulação de avanço temporal

2.8 Sugestões de Automações

Adicionalmente, a aplicação é capaz de sugerir automaticamente novas automações com base no comportamento observado ao longo do tempo, nomeadamente através de padrões associados a horários exatos de utilização dos dispositivos. O utilizador pode analisar estas sugestões e decidir se pretende ou não adicioná-las ao sistema.

A Figura 18 apresenta um exemplo destas sugestões, onde é possível visualizar a automação proposta e escolher se a mesma deve ser aceite ou rejeitada.

```
Sugestão 1
Tipo: AutomacaoEvento
Utilizador: Ana
Condição: Às 20:00
Ações:
  1. Desligar dispositivo L8
  2. Desligar dispositivo S4
Aceitar? (s/n):
```

Figura 18: Sugestão de automação baseada em padrões de utilização

Caso o utilizador aceite a sugestão, esta é automaticamente adicionada ao conjunto de automações da casa, passando a ser executada de forma automática sempre que as condições definidas se verificarem.

